MYTHIC

GEN2 User Guide

May 2026

Document Version 1.2
SDK Version v26.05.0

MYTHIC

1 Changelog

- Version 1.2
  - Updated to include changes in SDK 26.05
- Version 1.1
  - Added Changelog section to be consistent wth other Mythic Documents
  - Changed document version from 1.1-RC1 to 1.1
- Version 1.0
  - First version of the SDK v26.02.0 User Guide

2 Changes in SDK 26.05

- The `run_mythic_sdk_container.sh` and `gpu_run_mythic_sdk_container.sh` scripts have the following updates:
  - Directory paths can be overridden with environment variables. This allows the scripts to be used unmodified outside Cadence's collaboration chamber.
  - The default directory for artifact output is the user's home directory; previously, it was `/rscratch/tonbomythic3/${USERNAME}`.
- The `vnn-compiler-bin` and `mythic-vnn-compiler` components shipped in SDK 26.02 have been removed. These components are now integrated into the compiler container and the mythic-compiler application. A new `VNN` section has been introduced in the compiler config. A new `VNN` artifact is generated under the compiled artifact.
- mythic-ppa-estimators reports combined throughput for analog and digital NPUs. Each component runs sequentially, so that the combined latency is (`analog_npu_latency` + `digital_npu_latency`). This represents the worst-case latency for Mythic's NPU. Streaming execution, in which digital execution is performed in parallel with analog execution, will be introduced in Phase 3.5. This will reduce end-to-end latency.
- Model conversion and compilation have been updated to use ONNX opset=20.
- Model accuracy improves by 1% (model-dependent) due to the new analog settings.
- Two new models are introduced:
  - Added BEVFormer-tiny with an input resolution of 1600x900. Both a trained artifact and a compiler-ready artifact are provided. Accuracy simulation, compilation, and performance estimation are supported with this artifact. Note that, due to limitations in Mythic's functional simulator, performance estimates must be run with a single camera feed. The final latency must be estimated by multiplying the single-camera latency by six.
  - Added Pythia-70m trained artifact. Compilation and performance estimation are not supported. Compilation of LLMs will be supported in Phase 3.5 as part of InternVL3.

3 Introduction

This v26.05.0 SDK consists of the following components:

1. **Two Docker containers:**
   a. **Compiler** (`compilerd-bin`): Compiles pre-trained networks, predicts model latency, and estimates power consumption. Accessed via a Python frontend installed in the `mythic-sdk` container.
   b. **Mythic SDK** (`mythic-sdk`): Primary working environment. This includes the functional/accuracy simulator, example training code, pretrained models, and the compiler frontend interface.

2. **Supported models:** ResNet-50 (224 × 224), YOLOv8s Object Detection (1920x1080), YOLOv8s Pose (1920x1080), YOLOv8s Segmentation (1920x1088), YOLOPX (1280x720), BEVFormer-tiny (6x1600x900), and Pythia-70m. Each model includes an FP32-trained artifact and an ANA8-trained artifact. Compiler-ready artifacts are provided for all models except Pythia. Each of these artifacts can be regenerated by following the training guides included in the SDK documentation.

4 Prerequisites

- Log in to https://cloudburst.cadence.com
- You must be a member of `tonbomythic3` and `docker` groups. Verify that you are a member of the correct groups.

```
aw71ut01.cadence.com::~ > groups
ccusers docker tonbomythic3 aw71
```

- Verify you have access to the `/projects/tonbomythic3` directory

```
aw71ut01.cadence.com::~ > cd /projects/tonbomythic3/
aw71ut01.cadence.com::/projects/tonbomythic3 > ls
dataset/  tools/  users/
```

5 SDK Docker Containers

The v26.05.0 SDK is located in the `/projects/tonbomythic3/tools/sdk_26.05.0_containers` directory. The SDK is now distributed in three zip files: `compiler-installer-m2000-v26.05.0.zip`, `sdk-docker-image-installer-m2000-v26.05.0.zip`, and `training-models-installer-m2000-v26.05.0.zip`. The zip files have already been extracted to the `archive` subdirectory of the `sdk_26.05.0_containers` directory so no further steps are required in the collaboration chamber. To set-up a new system, copy the zip files to that system and extract them with `unzip -o <zip_filename>`.

The structure of the archive directory is show below:

```
archive
├── compiler
│   └── compiler_m2000.tar
├── install_compiler.sh
├── install_sdk_docker_image.sh
├── install_training_models.sh
├── models
│   └── training
│       ├── bevformer
│       │   ├── bevformer-tiny-1600x900-trained.onnx
│       │   ├── bevformer-tiny-1600x900-trained.tar.gz
│       │   └── bevformer-tiny-fp32-1600x900.onnx
│       ├── huggingface_classifiers
```

```
│       │   ├── resnet50_0.8098_opset11_224x224.onnx
│       │   ├── resnet50_imagenet_trained.onnx
│       │   └── resnet50_imagenet_trained.tar.gz
│       ├── pythia
│       │   ├── pythia_70m_fp32.onnx
│       │   └── pythia-70m-trained.onnx
│       ├── yolopx
│       │   ├── multiclass_yolopx_fp32.onnx
│       │   ├── yolopx_trained.onnx
│       │   └── yolopx_trained.tar.gz
│       └── yolov8
│           ├── yolov8s_trained.onnx
│           ├── yolov8s_trained.tar.gz
│           ├── yolov8s-fp32-1920x1920_t125_e432.onnx
│           ├── yolov8s-pose_1920x1088_trained.onnx
│           ├── yolov8s-pose_1920x1088_trained.tar.gz
│           ├── yolov8s-pose_relu_fp32_1920_0.61.onnx
│           ├── yolov8s-seg_1920x1088_trained.onnx
│           ├── yolov8s-seg_1920x1088_trained.tar.gz
│           └── yolov8s-seg_relu_resize_fp32_1920.onnx
```

The `installer_compiler.sh` and `install_sdk_docker_image.sh` scripts can be used for installing the respective Docker images onto the system. Note these scripts **must** be called from the `archive` directory. A helper script, `load_and_tag_docker_images.sh`, can be used to call each script.

6 Invoking a SLURM session

Docker commands are **ONLY** available in an interactive compute (SLURM) node. There are two types of SLURM session that be launched:

1. **CPU-only**: This should be used for compilation and running Mythic's PPA estimator.
2. **GPU**: This should be used for running Mythic's accuracy estimator. The GPU terminal should be shutdown when its not in use to avoid excessive charges.

The command for launching a **CPU-only** SLURM session is:

```
aw71ut01.cadence.com::~ > sbatch -p aw71-interactive -c 15 --wrap xterm --wait
```

The command for launching a **GPU** SLURM session is:

```
aw71ut01.cadence.com::~ > sbatch -p aw71-gpu-g6e --wrap=xterm --wait
```

Once the SLURM session has been activated, the SDK containers need to be installed. The process for doing this depends on whether a CPU-only or GPU session has been invoked. Correspondingly this document has been split into two sections. The first describes how to use the SDK on a CPU-only node and the second how to use the SDK on a GPU node.

7 Compilation and PPA Estimators

7.1 Loading Docker Images

First launch a CPU-only SLURM node as described above. Then check that v26.05.0 SDK containers are available using the `docker images` command. This is required if you have launched a new interactive compute node since you may get a node that does not already have the containers installed. The following repositories and tags should be seen.

```
 [125] ip-10-1-143-132.cadence.com ../sdk_26.05.0_containers > docker images
REPOSITORY                                    TAG              IMAGE ID       CREATED
SIZE
gcr.io/mythic-devops/mythic-sdk-ubuntu-24.04   m2000-v26.05.0   7fa5a22eb017   3 hours
ago   9.87GB
gcr.io/mythic-devops/compilerd-bin             1.5.2            86bac989d203   5 days
ago    1.75GB
```

There are two options installing the docker image. The first is to use the help script `load_and_tag_docker_images.sh`. This can be called from anywhere in the collaboration chamber.

```
/projects/tonbomythic3/tools/sdk_26.05.0_containers/load_and_tag_docker_images.sh
```

Alternatively call the installation scripts directly.

```
cd /projects/tonbomythic3/tools/sdk_26.05.0_containers
./install_compiler.sh
./install_sdk_docker_image.sh
```

7.2 Launching the Mythic SDK Container

A helper script, `run_mythic_sdk_container.sh`, is provided to launch the SDK container. This script sets environment variables for disabling huggingface network access and mounts directories for storing data. There are three types of data storage mounted:

- Temporary files that are created during compilation. These files are written to `/tmp/data` and are deleted once compilation or PPA estimation is successfully completed.
- Compiled files or PPA estimates. The docker mounts `$HOME` so that these files can be written to here. The `MYTHIC_WORKSPACE` environment variable can be used for overriding this.
- Datasets are mounted from `/projects/tonbomythic3/datasets`. The `DATASET_DIR` environment can be used for overriding this.

Launch the SDK container:

- ```
  p-10-1-143-42.cadence.com ../tonbomythic3> ./run_mythic_sdk_container.sh
  root@2e1150dde260:~/mythic_sdk/v26.05.0#
  ```

You must now enable the Python virtual environment for Mythic's SDK.

```
root@2e1150dde260:~/mythic_sdk/v26.05.0# source $MYTHIC_SDK_ROOT/mythic-model-zoo/venv/
bin/activate
(venv) root@2e1150dde260:~/mythic_sdk/v26.05.0#
```

7.3 Compilation

Once the mythic-sdk container is running the `mythic-compiler` can be used to create firmware artifacts. The syntax for compilation is shown below. The output artifact can be written to `/rscratch/tonbomythic3/$USER` in the host filesystem so that it persists across Docker sessions. Since the Docker container does not know `<your_username>` this must be manually added instead of using `$USER`.

```
mythic-compiler --input-artifact <compiler_ready_artifact> --compiler-config <model-
compiler-configuration> --output-artifact /rscratch/tonbomythic3/<your_username>/
<model_name>.tar.gz
```

The table below shows filenames for the compiler ready artifacts and compiler configuration for each model.

Model	Compiler ready artifact	Compiler Configuration
ResNet-50	`$MYTHIC_SDK_ROOT/models/training/huggingface_classifiers/resnet50_imagenet_trained.tar.gz`	• `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/huggingface_classifiers/compiler/resnet50_imagenet_224x224x3_m2024_default_optimization.yaml` • `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/huggingface_classifiers/compiler/resnet50_imagenet_224x224x3_m2024_high_optimization.yaml`
```

Model	Compiler ready artifact	Compiler Configuration
YOLOv8s Object Detection	`$MYTHIC_SDK_ROOT/models/training/yolov8/yolov8s_trained.tar.gz`	• `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/yolov8/compiler/yolov8s_1088x1920x3_m2024_default_optimization.yaml` • `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/yolov8/compiler/yolov8s_1088x1920x3_m2024_high_optimization.yaml`
YOLOv8s Pose	`$MYTHIC_SDK_ROOT/models/training/yolov8/yolov8s-pose_1920x1088_trained.tar.gz`	• `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/yolov8/compiler/yolov8s_pose_1088x1920x3_m2024_default_optimization.yaml` • `MYTHIC_SDK_ROOT/mythic-model-zoo/configs/yolov8/compiler/yolov8s_pose_1088x1920x3_m2024_high_optimization.yaml`
YOLOv8s Segmentation	`$MYTHIC_SDK_ROOT/models/training/yolov8/yolov8s-seg_1920x1088_trained.tar.gz`	• `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/yolov8/compiler/yolov8s_seg_1088x1920x3_m2024_default_optimization.yaml` • `MYTHIC_SDK_ROOT/mythic-model-zoo/configs/yolov8/compiler/yolov8s_seg_1088x1920x3_m2024_high_optimization.yaml`

Model	Compiler ready artifact	Compiler Configuration
YOLOPX	`$MYTHIC_SDK_ROOT/models/training/yolopx/yolopx_trained.tar.gz`	• `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/yolopx/compiler/yolopx_736x1280x3_m2048_default_optimization.yaml` • `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/yolopx/compiler/yolopx_736x1280x3_m2048_high_optimization.yaml`
BevFormer-tiny	`$MYTHIC_SDK_ROOT/models/training/bevformer/bevformer-tiny-1600x900-trained.tar.gz`	• `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/bevformer/compiler/bevformer_tiny_backbone_6x928x1600x3_m2072_default_optimization.yaml` • `$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/bevformer/compiler/bevformer_tiny_backbone_6x928x1600x3_m2072_high_optimization.yaml`

For example, to compile ResNet-50 the following syntax should be used. Similar syntax can be used for all other models.

```
mythic-compiler --input-artifact $MYTHIC_SDK_ROOT/models/training/
huggingface_classifiers/resnet50_imagenet_trained.tar.gz --compiler-config
$MYTHIC_SDK_ROOT/mythic-model-zoo/configs/huggingface_classifiers/compiler/
resnet50_imagenet_224x224x3_m2024_high_optimization.yaml --output-artifact /rscratch/
tonbomythic3/<your_username>/resnet50_compiled.tar.gz
```

The compiled artifact now contains reports for SRAM and weight utilization that can be extracted as follows:

```
tar -xOzf <archive.tar.gz> artifacts/firmware/weight_utilization.txt
tar -xOzf yolo-fw.tar.gz artifacts/firmware/sram_utilization.txt
```

7.4 Mythic PPA estimators

Mythic-ppa-estimators takes a compiled firmware artifact as its input and then outputs estimates of latency and power. The command line usage for `mythic-ppa-estimates` is shown below:

```
mythic-ppa-estimators --help
usage: mythic-ppa-estimators [-h] [--estimate-performance] [--estimate-power] [--power-
inference-rate POWER_INF_RATE] model_artifact_path

Run the Mythic PPA Estimators on a model artifact

positional arguments:
  model_artifact_path   Path to Mythic model artifact archive (tar.gz).

options:
  -h, --help            show this help message and exit
  --estimate-performance
                        Estimate performance for model artifact
  --estimate-power      Estimate power for model artifact
  --power-inference-rate POWER_INF_RATE
                        Set a target inference rate in frames/second (fps) for power
estimation
```

Here is an example of running the estimator on ResNet-50 at maximum inference speed.

```
~/mythic_sdk/v26.05.0# mythic-ppa-estimators --estimate-performance --estimate-power
resnet50_compiled.tar.gz

Successfully ending simulation.
INFO:mythic.ppa_estimators.estimate:Performing performance estimation on compiled
network...
✔️ Opening digital NPU JSON file: /work-dir/artifacts/firmware/vnn/MythicResnet50
✔️ Opening HDF5 file: /work-dir/perf_trace_dump.h5
✔️ Parsing /sram_accesses [7/7] host_interface_tile
✔️ Parsing /ace_calcs [6/6] ace_tile_05
✔️ Parsing /simd_calcs [7/7] host_interface_tile
✔️ Computing aggregate statistics
✔️ Estimating durations [2679/2679] (timestep=2678)
Analog NPU Total Estimated Processing Time for Compiled ONNX File: 0.44 ms
Analog NPU Estimated Frame Rate for Compiled ONNX File: 2,274.40 fps (frames per second)
Critical Path ACE Latency: 0.43 ms
Maximum SRAM Read/Write Time (over 6 ACE tiles): 0.44 ms
Maximum SIMD Operation Time (over 6 ACE tiles): 0.00 ms
```

```
Average SRAM Read/Write Time (averaged over 6 ACE tiles): 0.34 ms
Average SIMD Operation Time (averaged over 6 ACE tiles): 0.00 ms
Total Executed ACE Operations: 47,236
Total Executed ACE MACs: 5,324,275,712
Maximum Theoretical ACE Execution Time (With No Parallelization Using 1 ACE): 7.56 ms
Minimum Theoretical ACE Execution Time (With Even Parallelization Across 24 ACEs): 0.31
 ms
ACE Utilization (Minimum Theoretical Time/Estimated Processing Time): 71.62%
Total SRAM Bytes Read Across Chip: 140,795,352 bytes
Total SRAM Bytes Written Across Chip: 96,647,544 bytes
Estimated Die Area to Achieve Estimated Processing Time: 253.09 mm^2
✔️ Closing HDF5 file
----------------------------------------------------------------------
Digital NPU Performance Metrics (from separate ONNX graph processing)
  Total Estimated MACs Targeting Digital NPU: 2,000,000
  Digital NPU MAC Utilization: 30.00%
  Digital NPU Cores: 1
  Digital NPU Frequency: 1,000 MHz
  Digital Estimated Frame Processing Time: 0.11 ms
  Digital Estimated Frame Rate: 9,375.06 fps (frames per second)
----------------------------------------------------------------------
Combined Analog + Digital NPU Latency: 0.55 ms
Combined Analog + Digital NPU Total Estimated Frame Rate: 1,830.36 fps (frames per
second)

NOTE: The information reported above may include the Analog NPU or the Analog
NPU and a digital NPU as indicated. This estimated processing time and
associated frame rate only covers the execution of the compiled ONNX model(s)
running on one or both of those compute solutions in isolation. It does not
include PCIe transfer overhead. Performance estimation for additional processing
steps and a combined compute hardware solution will be added in future versions.

INFO:mythic.ppa_estimators.estimate:Performing power estimation on compiled network...
Process Nodes:  Analog 28nm, Digital 5nm
Number of ACEs: 24
Number of Digital NPU Cores: 1
Inference Rate: 1830 frames/second (fps)

Analog NPU Estimated Power: 1.104 W for target ONNX file
    Functional Unit Power: 0.821 W
    Interconnect Power:    0.283 W

Digital NPU Estimated Power: 0.009 W for digital target ONNX file

Total Combined (Analog + Digital NPU) Power: 1.113 W

NOTE: Estimation for leakage, clock tree, chip I/O power will be added in future
versions.
INFO:mythic.ppa_estimators.estimate:Estimator tools complete.
```

```
INFO:mythic.ppa_estimators.interface:Creating resnet_ppa_2026_05_30_00_56_13.tar.gz with
saved estimation log data.
```

8 Accuracy Simulation

8.1 Loading Docker Images

The GPU 's Linux image already has a number of Docker images loaded. This means there is insufficient space to load Mythic's SDK. To address this all of the extraneous images must be deleted. Note that since they are part of the GPU's Linux image they will return when the SLURM session is restarted. To delete these images, first use `docker images` to check they are loaded:

```
docker images
REPOSITORY          TAG                                IMAGE ID       CREATED         SIZE
nvidia/cuda         13.0.2-cudnn-runtime-ubi8          df28245ee1d1   5 weeks ago     3.2GB
nvidia/cuda         13.0.2-cudnn-runtime-rockylinux9   ed6de7937247   2 months ago
3.23GB
alpine              latest                             706db57fb206   2 months ago
8.32MB
nvidia/cuda         12.2.2-runtime-ubi8                d7f79f7d4ba5   2 years ago
2.33GB
nvidia/cuda         12.0.1-devel-ubi8                  531e1414a4db   2 years ago
7.16GB
nvidia/cuda         12.0.1-runtime-ubi8                c4c296079d5e   2 years ago
2.62GB
nvidia/cuda         12.1.1-runtime-ubi8                04abb8bd0201   2 years ago
2.39GB
pytorch/pytorch     2.0.0-cuda11.7-cudnn8-runtime      372afdac2915   2 years ago
6.44GB
```

If you see the above images they can be quickly deleted with the following command:

```
docker rmi `docker images -a -q`
```

There are two options installing the docker image. The first is to use the help script `load_and_tag_docker_images.sh`. This can be called from anywhere in the collaboration chamber.

```
/projects/tonbomythic3/tools/sdk_26.05.0_containers/load_and_tag_docker_images.sh
```

Alternatively, call the installation scripts directly.

```
cd /projects/tonbomythic3/tools/sdk_26.05.0_containers
./install_compiler.sh                    # The accuracy simulation does not need the
compiler so this can be skipped.
```

```
./install_sdk_docker_image.sh
```

8.2 Launching the Mythic SDK Container

A helper script, `gpu_run_mythic_sdk_container.sh`, is provided to launch the SDK container with special parameters needed for the GPU. This script sets environment variables for disabling huggingface network access and mounts directories containing datasets.

```
ip-10-1-143-139.cadence.com ../sdk_26.05.0_containers > ./
gpu_run_mythic_sdk_container.sh
root@ed3d5eafba20:~/mythic_sdk/v26.05.0#
```

This script mounts the `dataset` directory to `mythic-model-zoo/datasets`.

8.3 Running an Accuracy Simulation

To run the accuracy simulation we first need to set-up the environment for the model we want to test. This is done by sourcing a `.env` script in the `mythic-model-zoo/scripts` directory. The following table summarizes the scripts that available.

Model	Environment script
ResNet-50	`$MYTHIC_SDK_ROOT/mythic-model-zoo/scripts/huggingface_classifiers/resnet50-m2000.env`
YOLO8s Object Detection	`$MYTHIC_SDK_ROOT/mythic-model-zoo/scripts/yolov8/detect-m2000.env`
YOLO8s Pose Detection	`$MYTHIC_SDK_ROOT/mythic-model-zoo/scripts/yolov8/pose-m2000.env`
YOLO8s Segmentation	`$MYTHIC_SDK_ROOT/mythic-model-zoo/scripts/yolov8/segment-m2000.env`
YOLOPX	`$MYTHIC_SDK_ROOT/mythic-model-zoo/scripts/yolopx/yolopx-720p-m2000.env`

Model	Environment script
BEVFormer-tiny	`$MYTHIC_SDK_ROOT/mythic-model-zoo/scripts/bevformer/bevformer-tiny-1600x900-trained.onnx`
Pythia	`$MYTHIC_SDK_ROOT/mythic-model-zoo/scripts/pythia/pythia-70m-trained.onnx`

The Collaboration Chamber GPU has memory limitations that require reducing the number of data loader workers and the batch size. The commands below show how to run the accuracy simulation for each model. Note that the parameters used for setting the number of data workers and batch size vary depending on the underlying framework that is being used. In the case of YOLOv8 this is Ultralytics and in the case of ResNet-50 this is HuggingFace.

8.3.1 ResNet-50 Accuracy Simulation

```
cd $MYTHIC_SDK_ROOT/mythic-model-zoo
source  scripts/huggingface_classifiers/resnet50-m2000.env  # This script must be
sourced from mythic-model-zoo directory

DATASET=datasets/imagenet_huggingface/ python3 scripts/common/convert_model.py
steps=eval_trained trained_model=../models/training/huggingface_classifiers/
resnet50_imagenet_trained.onnx
```

8.3.2 YOLOv8s Object Detection Accuracy Simulation

```
cd $MYTHIC_SDK_ROOT/mythic-model-zoo

source scripts/yolov8/detect-m2000.env # This script must be sourced from mythic-model-
zoo directory

python3 scripts/common/convert_model.py steps=eval_trained trained_model=../models/
training/yolov8/yolov8s_trained.onnx eval_config.batch=1 +eval_config.workers=1
```

YOLOv8s accuracy simulation defaults to `mythic-model-zoo/datasets/coco` directory for its dataset location so there no need to use the DATASET variable. The eval_config.batch=1 (batch size = 1) and +eval_config.workers=1 (data loader workers = 1) are used to avoid out of memory errors in the collaboration chamber.

8.3.3 YOLOv8s Pose Accuracy Simulation

```
cd $MYTHIC_SDK_ROOT/mythic-model-zoo

source scripts/yolov8/pose-m2000.env

python3 scripts/common/convert_model.py steps=eval_trained trained_model=../models/
training/yolov8/yolov8s-pose_1920x1088_trained.onnx eval_config.batch=1
 +eval_config.workers=1
```

YOLOv8s Pose accuracy simulation defaults to `mythic-model-zoo/datasets/coco-pose` directory for its dataset location so there no need to use the DATASET variable. The eval_config.batch=1 (batch size = 1) and +eval_config.workers=1 (data loader workers = 1) are used to avoid out of memory errors in the collaboration chamber.

8.3.4 YOLOPX Accuracy Simulation

```
cd $MYTHIC_SDK_ROOT/mythic-model-zoo

source scripts/yolopx/yolopx-720p-m2000.env # This script must be sourced from mythic-
model-zoo directory

DATASET=datasets/bdd100k/ python3 scripts/common/convert_model.py steps=eval_trained
trained_model=../models/training/yolopx/yolopx_trained.onnx training_config.WORKERS=1
```

The training_config.WORKERS=1 sets the number of data loader works to one to avoid out-of-memory issues in the Collaboration Chamber.

8.3.5 BEVFormer

```
cd $MYTHIC_SDK_ROOT/mythic-model-zoo

source scripts/bevformer/bevformer-tiny-1600x900.env # This script must be sourced from
mythic-model-zoo directory


DATASET=datasets/nuScene/ python3 scripts/common/convert_model.py steps=eval_trained
trained_model=../models/training/bevformer/bevformer-1600x900-trained.onnx
```

Note, that nuScenes requires that annotations are generated as described in the BEVFormer training guide. Mythic has been unable to generate the annotations in Collaboration Chamber due to directory permissions.

8.4 Datasets

All the datasets are stored in `/projects/tonbomythic3/datasets` and mounted in the SDK Container in `$MYTHIC_SDK_ROOT/datasets`. The datasets are shown below

```
datasets/
├── bdd100k                 # BDD100K dataset for YOLOPX
├── BDD100K.zip             # BDD100K dataset zip file
├── coco                    # COCO dataset for YOLOv8
├── coco-pose               # COCO-Pose dataset for YOLOv8-Pose
├── imagenet_huggingface    # ImageNet in Apache Arrow Format for ResNet-50
├── imagenet_subset         # Subset of ImageNet. Useful for quick tests
├── nuScene                 # nuScene dataset for BevFormer
```

9 Weight Estimation

weight_estimation.py takes a trained onnx file as its input and then outputs the weight residuals for a typical ACE. The residuals will be stored as a dictionary in a pickle file. The dictionary keys will be the name of the layer and the values will be a numpy array.

The command line usage for `weight_estimator` is shown below:

```
python3 -m mythic.acm.denali.weight_estimator  yolopx_trained.onnx  residuals.pkl
```

To get the weights and residuals in the same shape and scale:

```
weights = (weights/128).reshape(weights.shape[0], np.prod(weights.shape[1:])).T
```